CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 19: Caches II

Instructor: Justin Yokota

Agenda

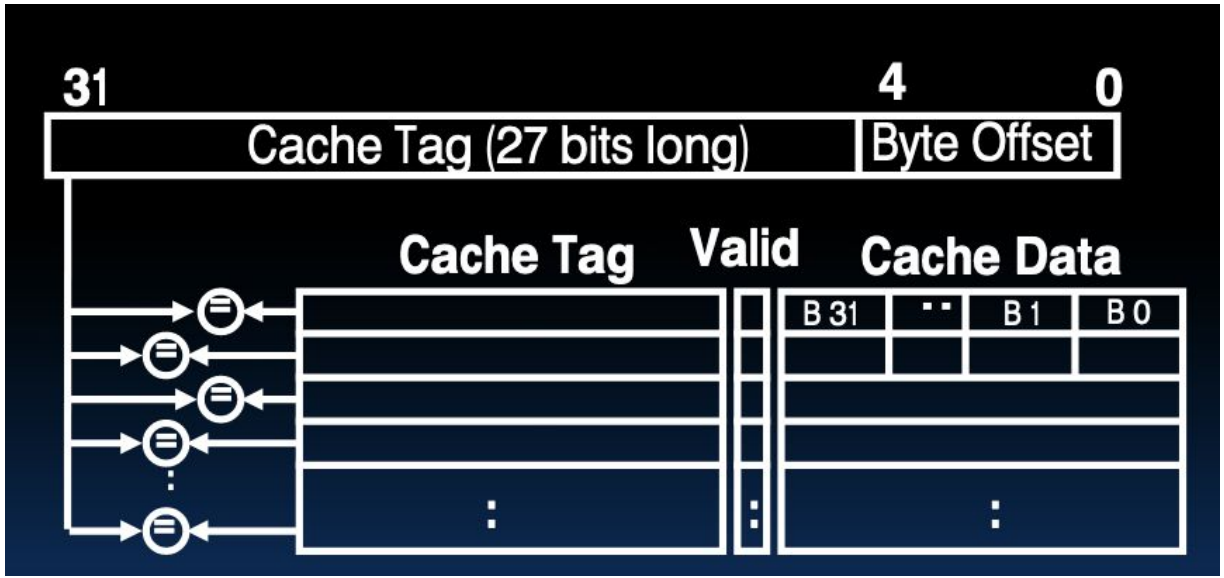
- Direct-Mapped Caches
- N-way Set Associative Caches
- Miss Classifications
- Multilevel Caches

Agenda

- **Direct-Mapped Caches**
- N-way Set Associative Caches
- Miss Classifications
- Multilevel Caches

Problems with Fully-Associative Caches

- While nice, an actual implementation of a fully associative cache has certain problems:
  - Hard to find a good and efficient eviction policy
  - Requires a lot of comparators/circuitry



Problems with Fully-Associative Caches

- While nice, an actual implementation of a fully associative cache has certain problems:
  - Hard to find a good and efficient eviction policy
  - Requires a lot of comparators/circuitry
- Main problem: any slot in our cache could store our data, so we need to check all spots
- Library analogy: We don't have any organization scheme in our bookshelf, so it takes a long time to check if our data is in our bookshelf
- In practice, hit time suffers
- Solutions?

Problems with Fully-Associative Caches

- Attempt #1: Sort the data in the cache, so we can check in log time.
  - Works if you have a bookshelf at home.
  - Doesn't work in caches.
  - Takes time to move data between blocks, so we don't want to have to do that (it's the same reason why we can't sort by LRU)
  - All the comparators still need to exist, and they run in constant time anyway (due to hardware parallelism)
- Attempt #2: Separate books into groups and only assign books to a cache block matching their group
  - Library analogy: Have the top shelf for books that start with "A", the second shelf for books that start with "B", etc. That way, we only need to check 1/26 of our bookshelf.
  - May cause unnecessary evictions if we don't have an even distribution of books
    - Not many books will start with "X", for example, so that space will likely go unused

How should we "group" our cache blocks into (for example) 8 groups?

- Hash the tag with some hashing algorithm
- Take the top three bits of the tag
- Take the bottom three bits of the tag
- Hash the data in the block with some hashing algorithm
- Take the first three bits of the data in the block
- Take the last three bits of the data in the block

How should we "group" our cache blocks into (for example) 8 groups?

- Hash the tag with some hashing algorithm
- Take the top three bits of the tag
- Take the bottom three bits of the tag
- Hash the data in the block with some hashing algorithm
- Take the first three bits of the data in the block
- Take the last three bits of the data in the block

How should we "group" our cache blocks into (for example) 8 groups?

Hash the tag with some hashing algorithm

Take the top three bits of the tag

Take the bottom three bits of the tag

Hash the data in the block with some hashing algorithm

Take the first three bits of the data in the block

Take the last three bits of the data in the block

Attempt 2: Picking a grouping mechanism

- No option that uses the data in the block will work
  - We need to get the block first to get the data, so we need to be able to figure out where the block is from the memory address alone
- Top three bits of the tag: Doesn't distribute blocks well
  - In practice, we'll be using consecutive blocks of memory, and chances are that consecutive blocks will have the same top three bits of memory address
- Hashing is a nice "all-purpose" approach to distributing data into even groups, but it's not as good here
  - Adds extra computation time
  - Distributes blocks randomly, whereas we can get exactly even distribution
- Bottom three bits of the tag: Works well with consecutive blocks of memory
  - Also scales to arbitrarily many cache slots without much additional circuitry
  - Technically also a hashing algorithm, just tag % 8.
  - Note: Assumes that we'll always have our number of groups be a power of 2.

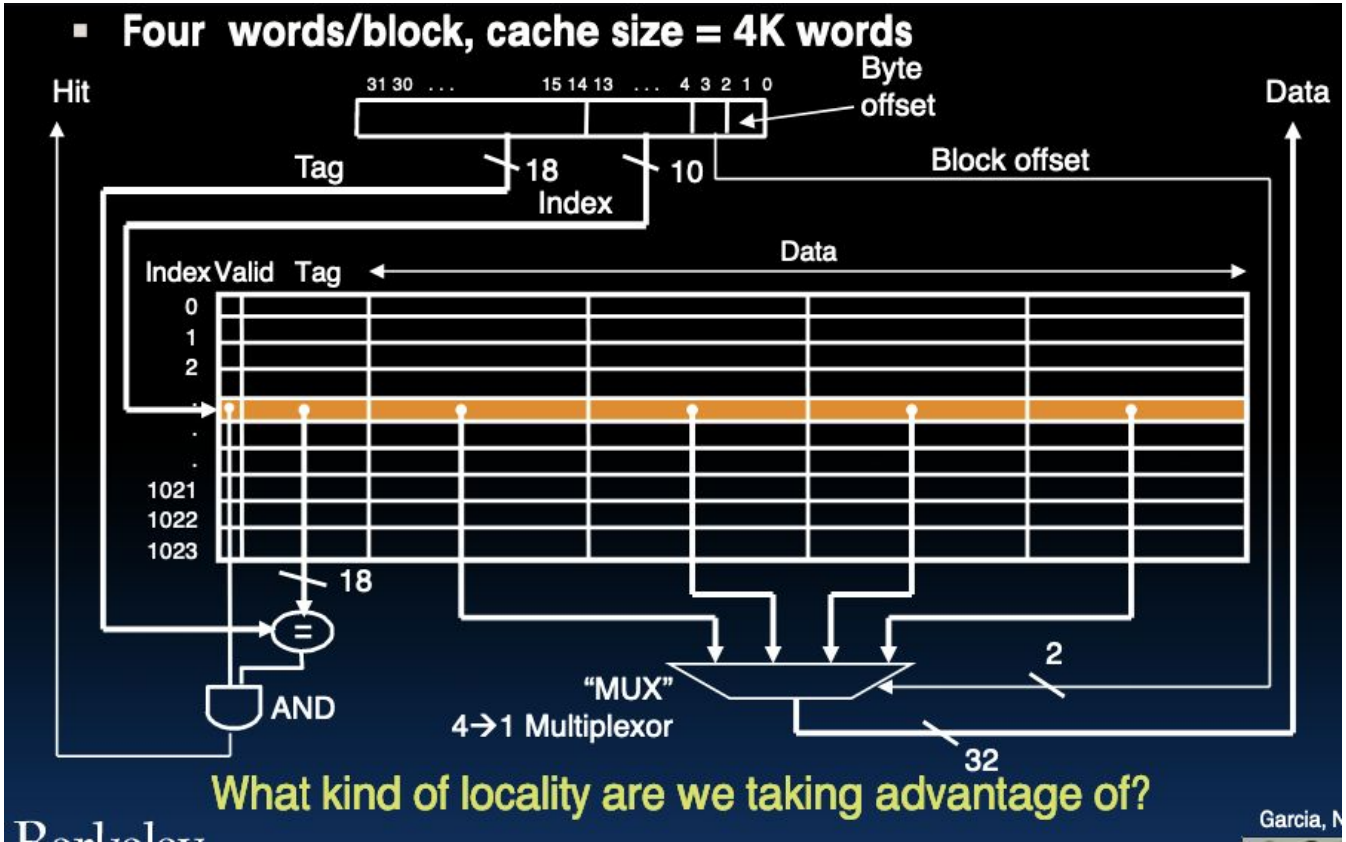
New Blocks

- Bottom three bits of the tag: Works well with consecutive blocks of memory
  - Also scales to arbitrarily many cache slots without much additional circuitry
- A memory address can be divided into a tag, an index, and an offset
- The bottom bits of our address is our offset, the next few bits the index, and the top bits will be the tag
- Ex. Let's say we have a block size of 4096 and 16 distinct groups
  - Ex. **`0xDEADBEEF`**
  - This address is in Block `0xDEADB`, which is the unique block with tag `0xDEAD` and index `0xB`, and is the `0xEEF`th byte in that block
- Determining the number of bits in each component of a cache address is called the "TIO breakdown", for "Tag, Index, Offset"

Direct-Mapped Cache (1/4)

- A direct-mapped cache is parametrized on two aspects: The block size, and the number of blocks that can be stored in the cache.
- Each slot in a direct-mapped cache is assigned a unique index; if we have 32 slots, one will be for index 0, one for index 1, … , and one for index 31
- When a memory access occurs:
  - Step 1: Check the unique slot that could contain our data. If the tag matches and the valid bit is on, cache hit.
  - Step 2: Otherwise, cache miss
- Library analogy: You buy a bookshelf that can hold N books. You set it up so that each slot can only hold a certain type of book, and promise never to put books in the wrong spot, even if it wastes space. That way, you only need to check one slot of your bookshelf when cache hitting.

Direct-Mapped Cache (2/4)



Direct-Mapped Cache (3/4): Example Memory Access

- Let's say we have a direct-mapped cache with 4 byte blocks and 4 blocks storage, and let's assume that we're working with a system that uses 10-bit addresses
- To access address 0x3CD:
  - Split into TIO:
    - 4 bytes blocks -> O = 2 bits
    - 4 indexes -> I = 2 bits
    - `0x3CD == 0b11 1100 11 01`
  - For index 3, the tag matches, and the valid bit is on, so we get the data
  - We get the 1st byte of the block, which is `0xDE`

Index	Tag	V	Data
0	0b10 0110	0	0xDE 0xAD 0xBE 0xEF
1	0b10 1100	0	0x01 0x23 0x45 0x67
2	0b01 1010	1	0xAB 0xAD 0xCA 0xFE
3	0b11 1100	1	0xAC 0xDE 0xFF 0x61

Direct-Mapped Cache (3/4): Example Memory Access

- Let's say we have a direct-mapped cache with 4 byte blocks and 4 blocks storage, and let's assume that we're working with a system that uses 10-bit addresses
- To access address 0x3C1:
  - Split into TIO:
    - 4 bytes blocks -> O = 2 bits
    - 4 indexes -> I = 2 bits
    - `0x3CD == 0b11 1100 00 01`
  - For index 0, the tag doesn't match, so it's a miss, even though the correct tag is in another index (same tag + different index is still a different memory address)

Index	Tag	V	Data
0	`0b10 0110`	1	`0xDE 0xAD 0xBE 0xEF`
1	`0b10 1100`	0	`0x01 0x23 0x45 0x67`
2	`0b01 1010`	1	`0xAB 0xAD 0xCA 0xFE`
3	`0b11 1100`	1	`0xAC 0xDE 0xFF 0x61`

Direct-Mapped Cache (4/4)

- Pros:
- Much simpler circuit
  - Instead of comparing everything, we mux all the indexes and use one comparator. Comparators use a lot of circuitry, so this is much smaller
  - Therefore leads to faster accesses
  - Also scales better than fully associative caches
- No need for an eviction policy
  - Only one block can be evicted, so no need to store LRU
- Cons:
- If an index isn't used, we end up with free space in our cache that we can't use
- Let's see how this behaves in Matrix Multiply with 4 blocks

Caching Example: Write-Back Direct-Mapped

- Start: Cache starts cold
  - Note: Index field, and no LRU field
    - Index isn't actually stored, but we'll show it anyway
  - Note: Matrix B has been transposed into Bt to optimize cache efficiency
- 0 misses, 0 hits

Index	Tag	V	Dirty	Data
0	0x100	0	0	0xBF 0x16 0x88 0x2B
1	0x133	0	0	0x3B 0x18 0xF1 0xB3
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back Direct-Mapped

- Access A[0]: 0x1000
  - Index 0, Tag 0x040 (taking the bottom two bits)
  - Miss, so replace the block in index 0

Index	Tag	V	Dirty	Data
0	0x100	0	0	0xBF 0x16 0x88 0x2B
1	0x133	0	0	0x3B 0x18 0xF1 0xB3
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back Direct-Mapped

- Access B[0]: 0x2000
  - Index 0, Tag 0x060
  - Miss, so replace the block in index 0
    - Uh oh

Index	Tag	V	Dirty	Data
0	0x040	1	0	0x01 0x02 0x03 0x04
1	0x133	0	0	0x3B 0x18 0xF1 0xB3
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back Direct-Mapped

- Access A[1]: 0x1001
  - Index 0, Tag 0x040
  - Miss, so replace the block in index 0
    - Oh no

Index	Tag	V	Dirty	Data
0	0x080	1	0	0x11 0x15 0x19 0x1D
1	0x133	0	0	0x3B 0x18 0xF1 0xB3
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back Direct-Mapped

- B[1]: Miss, A[2]: Miss, B[2]: Miss, … , C[0]: Miss
- 9 misses, 0 hits…
- A[0]: Miss, B[4]: Miss, but now address is 0x2010, which is index 1

Index	Tag	V	Dirty	Data
0	0x040	1	0	0x01 0x02 0x03 0x04
1	0x133	0	0	0x3B 0x18 0xF1 0xB3
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back Direct-Mapped

- A[1]: Hit, B[5]: Hit, … , C[1]: Miss, evicting Block 0x1000
- Overall for this element of C: 3 misses, 6 hits
    - Better!

Index	Tag	V	Dirty	Data
0	0x040	1	0	0x01 0x02 0x03 0x04
1	0x080	1	0	0x12 0x16 0x1A 0x1E
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

250			

Caching Example: Write-Back Direct-Mapped

- In general: if the element of C is on the diagonal, 9 misses, 0 hits
- Otherwise, 3 misses, 6 hits
- Total: 72 misses, 72 hits, or 50% hit rate
  - Back to pre-transpose hit rate!

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

Index	Tag	V	Dirty	Data
0	0x0C0	1	1	0xFA 0x04 0x?? 0x??
1	0x080	1	0	0x12 0x16 0x1A 0x1E
2	0x156	0	0	0xE6 0x57 0x49 0xEE
3	0x0E9	0	0	0xB5 0x81 0x67 0x3F

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

250			

Agenda

- Direct-Mapped Caches
- **N-way Set Associative Caches**
- Miss Classifications
- Multilevel Caches

Problem with Direct-Mapped Caches

- If we access two distinct segments of memory that are aligned, we end up continuously evicting blocks
  - This is known as **thrashing**, and causes a significant increase in misses
- Even without this, we have a number of unused slots in our cache until we start computing C[3], so until then, it's as if we had a smaller cache; we aren't using the cache to its full capacity
- Problem: How do we get the benefit of direct-mapped caches without the downsides?

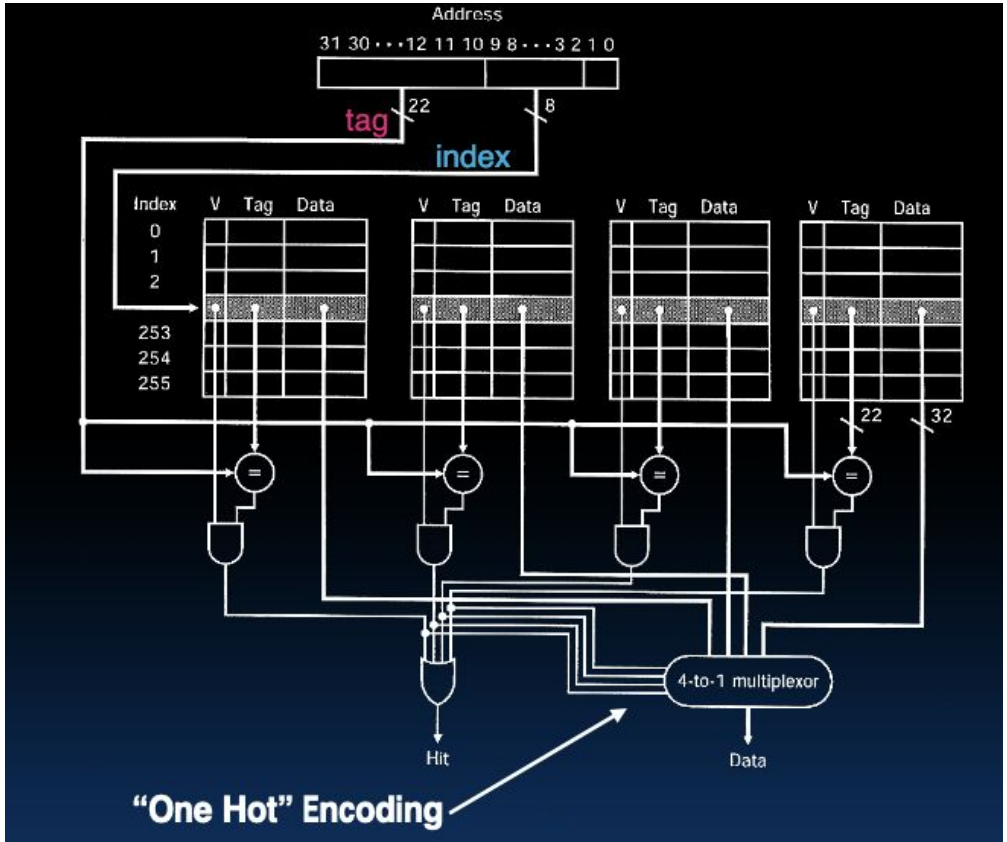
Meeting in the Middle: The Set-Associative Cache

- Direct-mapped caches thrash a lot, but fully associative caches use a lot of comparators
- Solution: Instead of just one slot per index, assign a small number of slots per index
- The **associativity** of a cache is the number of slots assigned to each index

N-way Set Associative Cache (1/4)

- An N-way set associative cache is parametrized on three aspects: The block size, the number of blocks that can be stored in the cache, and the associativity N.
- Blocks are split into groups of N, and each group of blocks is assigned a unique index
- When a memory access occurs:
  - Step 1: Check the slots that could contain our data. If the tag matches and the valid bit is on for any of these slots, cache hit.
  - Step 2: Otherwise, cache miss
- Library analogy: You buy a bookshelf. You set it up so that each shelf can only hold a certain type of book, and promise never to put books in the wrong shelf, even if it wastes space. That way, you only need to check a few blocks on a given shelf.

4-way Set Associative Cache (2/4)



N-way Set Associative Cache (3/4): Example

- Let's say we have a 2-way set-associative cache with 4 byte blocks and 4 blocks storage, and let's assume that we're working with a system that uses 10-bit addresses
- To access address 0x3CD:
  - Split into TIO:
    - 4 bytes blocks -> O = 2 bits
    - 4/2 = 2 indexes -> I = 1 bits
    - `0x3CD == 0b111 1001 1 01`
  - For index 1, the tag matches one of our blocks, and the valid bit is on, so we get the data
  - We get the 1st byte of the block, which is `0xDE`

I	Tag	V	Data
0	0b100 0110	1	0xDE 0xAD 0xBE 0xEF
0	0b101 1100	0	0x01 0x23 0x45 0x67
1	0b011 1010	1	0xAB 0xAD 0xCA 0xFE
1	0b111 1001	1	0xAC 0xDE 0xFF 0x61

N-way Set Associative Cache (4/4)

- Direct-mapped and Fully Associative caches can be considered special cases of the N-way set associative cache:
  - Direct-mapped == 1-way set associative
  - Fully associative == N = block count-way set associative
- Generally ends up with most of the performance of a fully associative cache and most of the circuit simplicity of a direct-mapped cache
- Let's see it in action with a 2-way set associative cache with 2 blocks

Caching Example: Write-Back LRU 2-way Set Associative

- Start: Cache starts cold
- A[0]: Miss, B[0]: Miss (but this time no eviction)
- 2 misses, 0 hits

I	Tag	LRU	V	D	Data
0	0x100	0	0	0	0xBF 0x16 0x88 0x2B
0	0x333	0	0	0	0x3B 0x18 0xF1 0xB3
1	0x156	0	0	0	0xE6 0x57 0x49 0xEE
1	0x0E9	0	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back LRU 2-way Set Associative

- A[1] Hit, B[1] Hit, … , B[3] Hit, C[0] Miss with eviction
  - Still have a bit of thrashing, but it's much less now
- Total: 6 Hits, 3 Misses
  - Same as Fully Associative cache!

I	Tag	LRU	V	D	Data
0	0x080	2	1	0	0x01 0x02 0x03 0x04
0	0x100	1	1	0	0x11 0x15 0x19 0x1D
1	0x156	0	0	0	0xE6 0x57 0x49 0xEE
1	0x0E9	0	0	0	0xB5 0x81 0x67 0x3F

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

C

Caching Example: Write-Back LRU 2-way Set Associative

- A[0] miss (evict block 0x200 with tag 0x100 and index 0), B[4] miss (index 1)
- A[1] - B[7] hit, C[1] hit
- Total for this element: 2 misses, 7 hits

I	Tag	LRU	V	D	Data
0	0x180	2	1	1	0xFA 0x?? 0x?? 0x??
0	0x100	1	1	0	0x11 0x15 0x19 0x1D
1	0x156	0	0	0	0xE6 0x57 0x49 0xEE
1	0x0E9	0	0	0	0xB5 0x81 0x67 0x3F

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back LRU 2-way Set Associative

- Computing C[2]: A[0] hits, B[8] misses, evicting block 0x300 … , C[2] misses
- Total for this element: 2 misses, 7 hits
- C[3]: Same as C[1], so 2 misses, 7 hits

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

I	Tag	LRU	V	D	Data
0	0x180	1	1	1	0xFA 0x04 0x?? 0x??
0	0x080	2	1	0	0x01 0x02 0x03 0x04
1	0x100	1	1	0	0x12 0x16 0x1A 0x1E
1	0x0E9	0	0	0	0xB5 0x81 0x67 0x3F

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

Caching Example: Write-Back LRU 2-way Set Associative

- C[4]: Similar to C[0], except this time, the A and C blocks are in index 1
  - 3 misses, 6 hits

I	Tag	LRU	V	D	Data
0	0x080	2	1	0	0x01 0x02 0x03 0x04
0	0x180	1	1	1	0xFA 0x04 0x0E 0x18
1	0x100	2	1	0	0x12 0x16 0x1A 0x1E
1	0x101	1	1	0	0x14 0x18 0x1C 0x20

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

250	260		

Caching Example: Write-Back LRU 2-way Set Associative

- C[5]: Acts like C[2] (B block on same index as A and C block)
  - 2 misses, 7 hits
- C[6],C[7]: 2 misses, 7 hits each

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

I	Tag	LRU	V	D	Data
0	0x100	1	1	0	0x11 0x15 0x19 0x1D
0	0x180	2	1	1	0xFA 0x04 0x0E 0x18
1	0x100	2	1	0	0x12 0x16 0x1A 0x1E
1	0x181	1	1	1	0x6A 0x?? 0x?? 0x??

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

250	260	270	280

Caching Example: Write-Back LRU 2-way Set Associative

- C[8]-C[11]: Same as first row
- C[12]-C[15]: Same as second row
- Total: 36 misses, 108 hits
- 75% hit rate; not as good as FA, but much better than DM!

I	Tag	LRU	V	D	Data
0	0x100	2	1	0	0x11 0x15 0x19 0x1D
0	0x101	1	1	0	0x13 0x17 0x1B 0x1F
1	0x081	2	1	0	0x05 0x06 0x07 0x08
1	0x181	1	1	1	0x6A 0x84 0x9E 0xB8

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

17	21	25	29
18	22	26	30
19	23	27	31
20	24	28	32

C

250	260	270	280

Set Associativity Conclusion

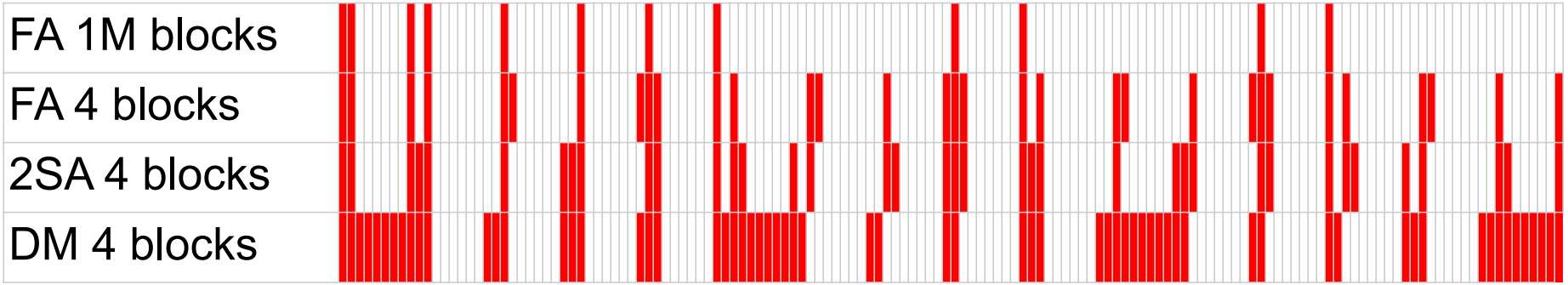
- Increasing associativity has pros and cons
- Pro:
  - Reduces thrashing (effect diminishes after a small associativity increase)
  - Increases cache usage
- Con:
  - Increases circuitry required
  - Forces a smaller cache, so the increased cache usage eventually gets countered by this anyway
- Goal is to get a good middle-ground between cache size and cache associativity
- How do we measure the effectiveness of our cache (relative to other cache types)?

Agenda

- Direct-Mapped Caches
- N-way Set Associative Caches
- **Miss Classifications**
- Multilevel Caches

Analyzing Cache Effectiveness

- Let's compare the hit patterns of a few cache types
- The below diagram shows the hit/miss pattern of various caches when run on the Matrix Multiply example shown in lecture
  - Red = Miss, White = Hit
- Also one more cache: A fully associative cache with an absurd number of blocks (an "ideal" cache which will hit as often as possible)



Analyzing Cache Effectiveness

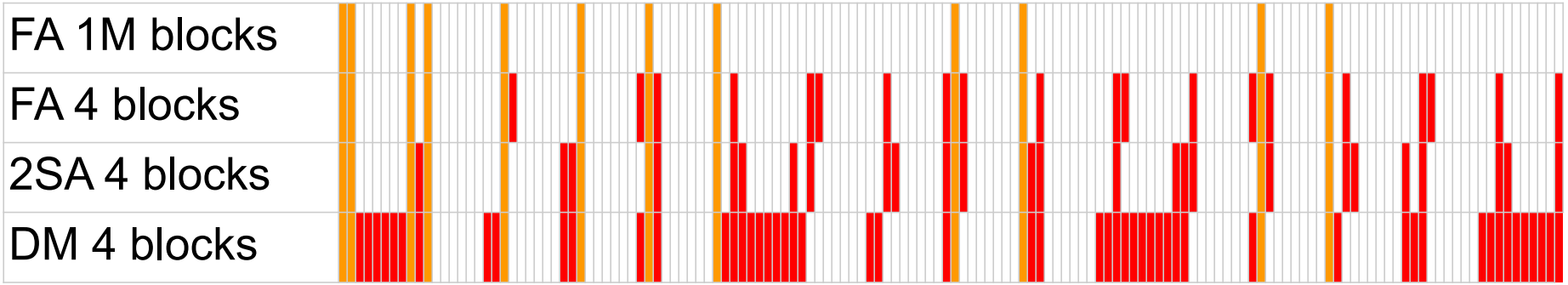
- When looking at cache hit/miss patterns, there were several interesting patterns
  - When we accessed a new set of cache blocks, there were significantly more misses than usual
  - When associativity increased, many misses were converted into hits, though a few hits turned into misses
- If we wanted to improve our cache structure, we'd need to know why misses happened
  - Do we increase associativity (and maybe decrease size to keep a constant hit time), or increase cache size (and maybe decrease associativity to keep a constant hit rate)?
- Goal: Classify misses according to why they occurred, so that we can figure out how we might change our cache to improve hit rate
  - Program-independent analysis, so we'll restrict to statements that can be made just off the cache state.

Miss Classifications: Definitional Goals

- Ideally, we have three main types of misses:
  - Edge cases cause this ideal to be impossible
- Compulsory Misses: Misses that are impossible to avoid; regardless of how we set up our cache
- Capacity Misses: Misses that were the result of our cache having limited space (not enough capacity)
- Conflict Misses: Misses that were added as a result of a low associativity/thrashing (blocks conflicted)

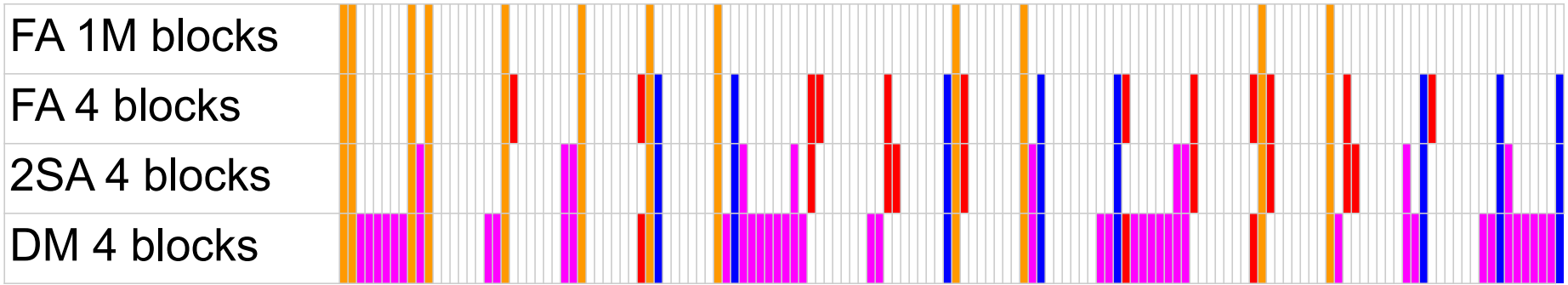
Compulsory Miss

- Misses that are impossible to avoid; regardless of how we set up our cache
- All of these are the first access to a block
  - Ex. Accessing A[0] for the first time
- Guaranteed to be misses, so there's no real way to prevent these (beyond increasing block size).
  - If most misses are compulsory, we can't improve hit rate much.



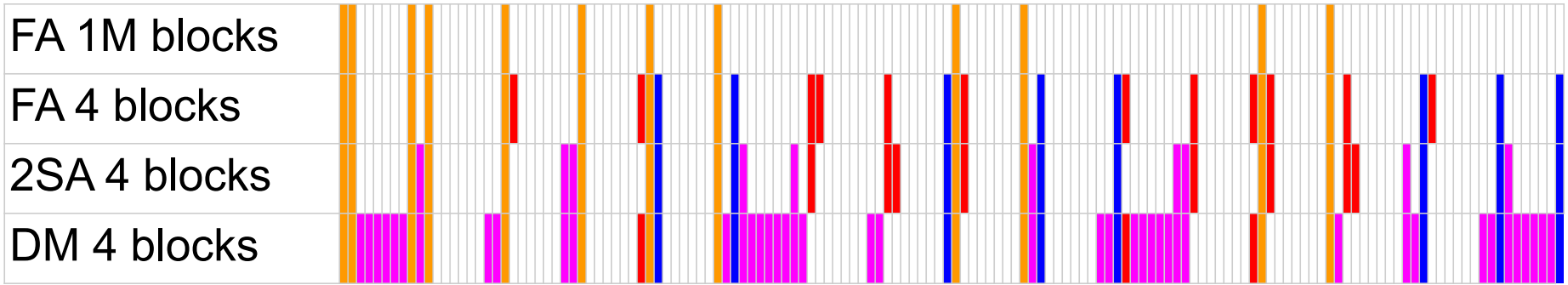
Conflict Miss

- Misses that were added as a result of a low associativity/thrashing (blocks conflicted)
- Intended to be misses that were added by decreased associativity
- https://bitsavers.org/pdf/dec/tech_reports/WRL-TN-53.pdf defines them as "misses that would not occur if the cache was fully-associative and had LRU replacement" (with all other noncompulsory misses being capacity)



Conflict Miss

- The problem with this definition is that it assumes that all misses in the Fully Associative cache also cause misses in lower associativities
  - This isn't actually true; as we see below, sometimes we get "conflict hits" because a block that would have been evicted ends up staying in the cache longer
  - Also requires a lot more work to compute, so we'll use an easier definition in this class



Miss Classifications: Definitions in this class

- Compulsory Misses: Misses that would occur even if the cache was a fully associative cache with infinite space
- Noncompulsory Misses: Misses that are not compulsory
- Let's look again at the 2-way set associative cache example

Caching Example: Hit Classifications

- Start: Cache starts cold
  - We'll restrict the cache to only the parts that are important (no data/dirty/etc, block instead of tag).
  - Also will write the status of each block on the right
  - Blocks will be ordered by LRU within a set (unlike an actual cache)
- First miss: A[0]
  - Compulsory miss

Block	Status	Block	Status
A[0]	NA	B[8]	NA
A[4]	NA	B[12]	NA
A[8]	NA	C[0]	NA
A[12]	NA	C[4]	NA
B[0]	NA	C[8]	NA
B[4]	NA	C[12]	NA

Index	Block	Index	Block
0	-	1	-
0	-	1	-

Caching Example: Hit Classifications

- Next miss is B[0]
  - Compulsory Miss
- Next miss is C[0]
  - Compulsory Miss, evicts A[0]
  - A[0] was evicted

Block	Status	Block	Status
A[0]	In Cache	B[8]	NA
A[4]	NA	B[12]	NA
A[8]	NA	C[0]	NA
A[12]	NA	C[4]	NA
B[0]	NA	C[8]	NA
B[4]	NA	C[12]	NA

Index	Block	Index	Block
0	A[0]	1	-
0	-	1	-

Caching Example: Hit Classifications

- A[0]
  - Noncompulsory miss
  - Evicts B[0]
- B[4]
  - Compulsory miss
- B[8]
  - Compulsory miss
  - Evicts C[0]

Index	Block	Index	Block
0	B[0]	1	–
0	C[0]	1	–

Block	Status	Block	Status
A[0]	Removed	B[8]	NA
A[4]	NA	B[12]	NA
A[8]	NA	C[0]	In Cache
A[12]	NA	C[4]	NA
B[0]	In Cache	C[8]	NA
B[4]	NA	C[12]	NA

Caching Example: Hit Classifications

- C[2]
  - Noncompulsory miss
  - Evicts A[0]
- A[0]
  - Noncompulsory miss
  - Evicts B[8]
- B[12]
  - Compulsory miss

Block	Status	Block	Status
A[0]	In Cache	B[8]	In Cache
A[4]	NA	B[12]	NA
A[8]	NA	C[0]	Removed
A[12]	NA	C[4]	NA
B[0]	Removed	C[8]	NA
B[4]	In Cache	C[12]	NA

Index	Block	Index	Block
0	A[0]	1	B[4]
0	B[8]	1	-

Caching Example: Hit Classifications

- A[4]
  - Noncompulsory miss
  - Evicts B[4]
- B[0]
  - Noncompulsory miss
  - Evicts C[0]
- C[4]
  - Compulsory miss
  - Evicts B[12]

Block	Status	Block	Status
A[0]	In Cache	B[8]	Removed
A[4]	NA	B[12]	In Cache
A[8]	NA	C[0]	In Cache
A[12]	NA	C[4]	NA
B[0]	Removed	C[8]	NA
B[4]	In Cache	C[12]	NA

Index	Block	Index	Block
0	C[0]	1	B[4]
0	A[0]	1	B[12]

Caching Example: Hit Classifications

- B[4]
  - Noncompulsory miss
  - Evicts A[4]
- A[5]
  - Noncompulsory miss
  - Evicts C[4]
- C[5]
  - Noncompulsory miss
  - Evicts B[4]

Block	Status	Block	Status
A[0]	In Cache	B[8]	Removed
A[4]	In Cache	B[12]	Removed
A[8]	NA	C[0]	Removed
A[12]	NA	C[4]	In Cache
B[0]	In Cache	C[8]	NA
B[4]	Removed	C[12]	NA

Index	Block	Index	Block
0	A[0]	1	A[4]
0	B[0]	1	C[4]

How to Improve your Cache Hit Rate

- Look at what type of miss is most common
  - Different access patterns will have different miss patterns, so test some "canonical" access sequence, like matrix multiplication
- If Compulsory misses are most common
  - Not really much you can do. Focus on something else to optimize
- If Capacity misses are most common
  - You're using too much memory at once for your cache to be efficient.
  - From the hardware perspective, increase cache size
  - From the software perspective, reduce the "working block"; try to split your code into parts that access only as much memory as your cache can store
- If Conflict misses are most common
  - Your cache needs more associativity
  - From the hardware side, increase associativity
  - From the software side, reduce the number of distinct chunks of memory you're accessing (though conflict misses tend to be minimal in most real-life caches)
- This class won't distinguish between capacity and conflict misses (though previous versions of this class might include questions on the topic)

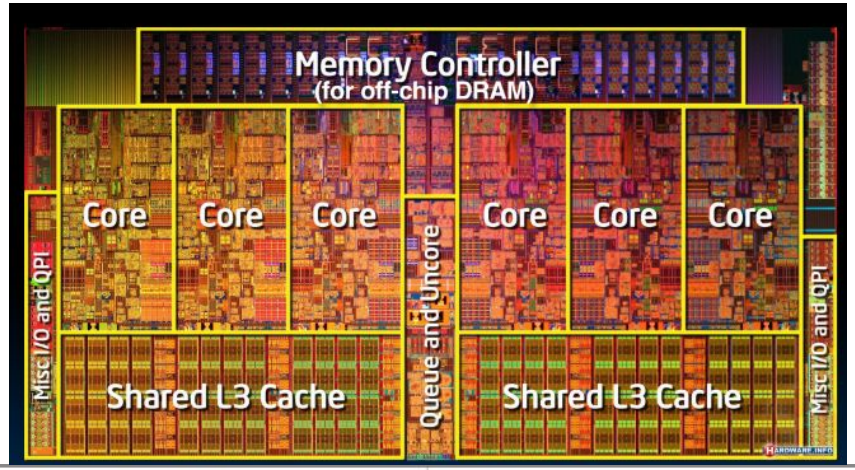
Agenda

- Direct-Mapped Caches
- N-way Set Associative Caches
- Miss Classifications
- **Multilevel Caches**

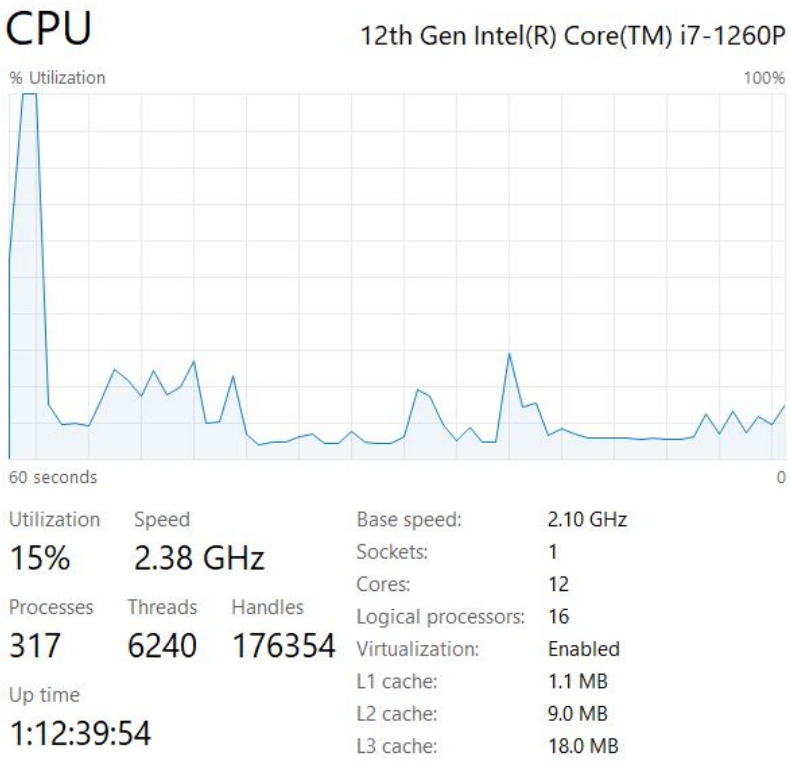
Multilevel Caches

- Current setup: If cache hit, then fast access. If cache miss, then slow access
- Problem: Hit rate scales with access time; the higher the hit rate we want, the slower we'll end up making cache accesses. With just one cache, we need to make some tough trade-offs
- Solution: Add multiple layers of caches.
  - Check L1 cache. If hit, we're done
  - If miss, check L2 cache. If hit, we're done, and bring data up to L1 cache
  - If miss, check L3 cache. If hit, we're done, and bring data up to L1 and L2 cache
  - …
- Each level of cache is larger than the previous and should have a subset of the data in the next cache
- Library analogy: We buy a bookshelf for right next to us, then we buy a larger bookshelf to put in our garage, then make a shed to store more books.

Multilevel Caches



L1 Cache Per P-Core	80 KB
L2 Cache Per P-Core	1280 KB
L1 Cache Per E-Core	96 KB
L2 Cache Shared by E-Cores	2 × 2 MB
L3 Cache Shared	18 MB



Multilevel Caches

Memory	Size	Latency	Bandwidth
L1 cache	32 KB	1 nanosecond	1 TB/second
L2 cache	256 KB	4 nanoseconds	1 TB/second Sometimes shared by two cores
L3 cache	8 MB or more	10x slower than L2	>400 GB/second
MCDRAM		2x slower than L3	400 GB/second
Main memory on DDR DIMMs	4 GB-1 TB	Similar to MCDRAM	100 GB/second
Main memory on Cornelis* Omni-Path Fabric	Limited only by cost	Depends on distance	Depends on distance and hardware
I/O devices on memory bus	6 TB	100x-1000x slower than memory	25 GB/second
I/O devices on PCIe bus	Limited only by cost	From less than milliseconds to minutes	GB-TB/hour Depends on distance and hardware

Multilevel Caches

- Generally, the L1 cache is small, but also on the CPU core
  - Each core gets its own L1 cache
  - Fast access, but also fairly small
- L2 cache is larger and slightly slower, but usually a bit farther away
  - Sometimes shared, sometimes on the core
- L3 cache is normally much larger and much slower, and is shared with all cores
- Using the numbers from the previous page as examples:
  - L1 hit time = 1 ns
  - L2 hit time = 1 ns + 4 ns = 5 ns
  - L3 hit time = 1 ns + 4 ns + 40 ns = 45 ns
  - L3 miss = 1 ns + 4 ns + 40 ns + 80 ns = 125 ns
- Miss rate is calculated only for accesses that "reach" that cache
  - L2 hit rate is % of L2 accesses that hit, so L1 hits don't count as L2 hits OR L2 misses

AMAT in Multilevel Caches

- Let's say we have a memory system with the following properties. What would be the AMAT of this system?
- Respond at PollEv.com/jy314

	Hit time	Hit rate
L1 Cache	1 ns	50%
L2 Cache	4 ns	75%
L3 Cache	40 ns	80%
SRAM	80 ns	100%

What is the AMAT of the cache in the previous slide?

Top

AMAT in Multilevel Caches

- Let's say we have a memory system with the following properties. What would be the AMAT of this system?
- 50% of accesses are L1 hits
- 50% of accesses are L1 misses
  - 50%*75% = ⅜ are L2 hits
- ⅛ of accesses are L2 misses
  - ⅛*80% = 1/10 are L3 hits
- 1/40 of accesses are L3 misses
  - And therefore access SRAM
- Total: 1 ns + 0.5*4 ns + ⅛ * 40 ns + 1/40 * 80 ns
- 10 ns access time = 8x speedup

	Hit time	Hit rate
L1 Cache	1 ns	50%
L2 Cache	4 ns	75%
L3 Cache	40 ns	80%
SRAM	80 ns	100%

Conclusion

- Caching is a powerful tool that lets us save memory access time
- Tons of different ways to customize your cache, with different advantages/disadvantages
  - Direct-mapped to Fully Associative
  - Eviction policy
  - Write-back/Write-through
  - Multi-level caching
  - Shared vs independent caches
- Hardware design ends up trying to pick the best parameters so that the cache works well for most programs
- Software design can be improved by knowing your cache system and writing code that will be efficient.